



МГТУ им. Н.Э. Баумана

Кафедра «Теоретическая информатика
и компьютерные технологии»

Клеточные автоматы в анализе формальных языков

Руководитель:
Непейвода А.Н.
a_nevod@mail.ru

Кочетков Д.И.
dert_vo@mail.ru

- Возросшее применение анализа формальных языков в прикладных задачах (например, LLM)
- Развитие платформ, поддерживающих массовый параллелизм
- Развитие альтернативных методов анализа формальных языков



Часть 1: быстрое введение



Регулярные языки

Определение

Регулярные языки — класс формальных языков, рекурсивно

- Пустое множество, язык из пустого слова, слово из одного символа алфавита — регулярные языки
- Конкатенация регулярных языков — регулярный язык
- Объединение регулярных языков — регулярный язык
- Замыкание Клини регулярного языка — регулярный язык
- Других регулярных языков нет

1. Задаются: регулярными выражениями, НКА (Построение Томпсона), ДКА(алгоритм детерминизации), грамматиками с правым выводом
2. Существует алгоритм минимизации, теорема Майхилла-Нероде, лемма о накачке и другие



Определение

Контекстно-свободные языки — класс формальных языков, порождаемые контекстно-свободными грамматиками

1. Задаются: автоматами с магазинной памятью(PDA), контекстно-свободными грамматиками
2. Существует лемма о накачке, теорема Майхилла-Нероде в ограниченном виде
3. Имеет деление на детерминированные и недетерминированные языки



Контекстно-зависимые языки

- Задаются контекстно-зависимыми грамматиками

Рекурсивно-перечислимые языки

- Задаются завершающимся алгоритмом распознавания для машины Тьюринга или другого Тьюринг-полного исполнителя
- Имеют в себе большое число классов сложности распознавания



Линейные контекстно-свободные языки — языки, распознаваемые линейными грамматиками с правилами вида

$$A_i \rightarrow uA_jv, A_x \in N, u, v \in \Sigma^* \quad A_i \rightarrow x_i, x_i \in \Sigma$$

- пример: язык палиндромов
- есть лемма о накачке



Линейно-конъюнктивные языки

Линейно-конъюнктивная грамматика задается кортежем

$G = (\Sigma, N, P, S)$, где

- Σ — множество символов входного алфавита. Далее в контексте грамматики оно будет называться множеством терминалов;
- N — множество нетерминальных символов, которые могут раскрываться в последовательность терминальных и нетерминальных символов;
- P — множество правил раскрытия нетерминалов вида $A \rightarrow \alpha_1 \& \alpha_2 \& \dots \& \alpha_n$, где $\alpha_i = \varphi_{i,1} A_i \varphi_{i,2}$ или $\alpha_i = \varphi_{i,0}$, $\varphi_{i,j} \in \Sigma^*$, $A_i \in N$;
- $S \in N$ — начальный символ.

Говорят, что слово порождается линейно-конъюнктивной грамматикой, если есть последовательность подстановок нетерминальных символов, которые позволяют породить данное слово. При этом в отличие от классической

контекстно-свободной грамматики, конъюнктивные правила

раскрытия (символ $\&$) должны выполняться все.



Часть 2: универсальный решатель?



Треллис автоматы

Определение

Треллис Автомат (далее — ТА) определяется кортежем $(Q, \Sigma, \delta, I, F)$, где

- Q, Σ — конечные множества состояний и символов входного алфавита ТА;
- $\delta : Q \times Q \rightarrow Q$ — правила перехода или локальная функция ТА;
- $I : \Sigma \rightarrow Q$ — отображение символов входного алфавита в состояние ТА;
- $F \subseteq Q$ — множество принимающих состояний.

Пусть $w = a_0 \dots a_n \in \Sigma^*$ - входная строка. Треллис автомат принимает w , если существует последовательность строк

$s_i = r_0 \dots r_{n-i} \in Q^*$, где

$s_0 = I(w) \quad s_{i+1} = r'_1 \dots r'_{n-i-1} : r'_j = \delta(r_j, r_{j+1}) \quad s_n \in F$



Треллис автомат для ПСП

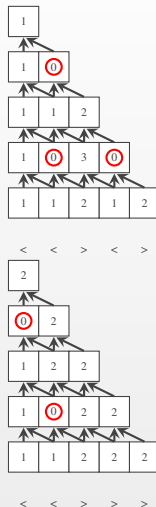
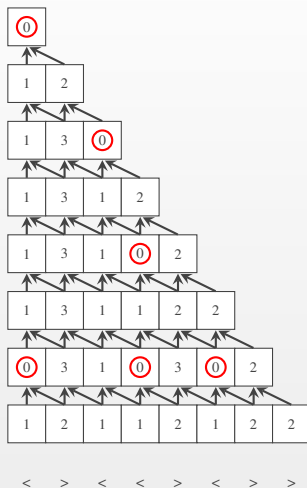
$$\Sigma = \{<, >\} \quad Q = \{0, 1, 2, 3\}$$

$$I(<) = 1 \quad F = \{0\}$$

$$I(>) = 2$$

	0	1	2	3
0	3	3	2	1
1	1	1	0	1
2	3	3	2	3
3	2	3	2	3

- 0 — ПСП
- 1 — префикс ПСП
- 2 — суффикс ПСП
- 3 — прочие строки



Свойства треллис автоматов

- Класс языков треллис автоматов замкнут относительно булевых операций над множествами, обращения, инъективных морфизмов, обратных морфизмов
- Во время вычисления ответа для слова ТА дает ответ о принадлежности языку и всех его подслов
- Пара состояний xu в конфигурации вычислений характеризуют подстроку awb , $a, b \in \Sigma$, $w \in \Sigma^*$, при этом x характеризует подстроку aw , а состояние y — wb .



- Алгоритм Охотина для линейно-конъюнктивных грамматик
- Упрощенный алгоритм для ДКА
- Алгоритм Охотина для Visibly Pushdown Automata
- Анализ сигналов
- Итеративное обучение КА



Теорема Охотина об эквивалентности

Для каждой линейно-конъюнктивной грамматики $G = (\Sigma, N, P, S)$ существует и может быть эффективно построен ТА $M = (\Sigma, Q, I, \delta, F)$ такой, что $L(M) = L(G) \setminus \{\varepsilon\}$. Для каждого ТА M существует и может быть эффективно построена линейно-конъюнктивная грамматика G такая, что $L(G) = L(M)$.¹

Грамматика $\rightarrow$ ТА

$$\left\{ \begin{array}{l} \text{ТА} \rightarrow \text{грамматика} \\ N = \{A_q \mid q \in Q\} \\ S \rightarrow A_q : \forall q \in F \\ A_{I(a)} \rightarrow a : \forall a \in \Sigma \\ A_{\delta(q_1, q_2)} \rightarrow A_{q_1} c \& b A_{q_2} : \\ \quad (q_1, q_2 \in Q, b, c \in \Sigma) \end{array} \right. \quad \left\{ \begin{array}{l} Q = \Sigma \times 2^N \times \Sigma \\ I(a) = (a, \{X \in N : X \rightarrow a\}, a) \\ \delta((a_1, X_1, b_1), (a_2, X_2, b_2)) = (a_1, X_3, a_2) \\ X_3 = \{A \mid \exists A \rightarrow a_1 B_1 \& \dots \& a_1 B_m \& \\ \quad \& A_1 b_2 \& \dots \& A_n b_2 : B_i \in X_2, A_j \in X_1\} \\ F = \{(a, X, b) \mid S \in X\} \end{array} \right.$$

¹ *Okhotin A.* On the equivalence of linear conjunctive grammars and trellis automata.



Пример преобразования грамматики в ТА

$$S \rightarrow aSa \mid b$$

В линейной нормальной форме

$$S \rightarrow aP \mid b; P \rightarrow Sa$$

Стартовые состояния

$$I(a) = (a, -, a) \quad I(b) = (b, S, b)$$

$$0 \mid (a, \{S, P\}, a)$$

$$1 \mid (a, \{P\}, a)$$

$$2 \mid (b, \{P\}, a)$$

$$3 \mid (a, -, b)$$

$$4 \mid (b, \{S\}, b)$$

$$5 \mid (b, -, b)$$

$$6 \mid (a, -, a)$$

$$7 \mid (a, \{S\}, a)$$

$$8 \mid (b, -, a)$$

	0	1	2	3	4	5	6	7	8
0	0	0	0	3	3	5	1	1	1
1	7	7	7	3	3	5	6	6	6
2	8	8	8	5	5	5	8	8	8
3	7	7	7	3	3	5	6	6	6
4	2	2	2	5	5	5	2	2	2
5	5	5	5	5	5	5	5	5	5
6	7	7	7	3	3	5	6	6	6
7	0	0	0	3	3	5	1	1	1
8	8	8	8	5	5	5	8	8	8



Построение из ДКА

Пусть $A = (Q, \Sigma, \delta, q_0, F)$ — детерминированный конечный автомат, тогда ТА определяется как

$$Q' = Q \times \Sigma$$

$$I(a) = \delta(q_0, a)$$

$$\delta'((x, a), (y, b)) = (\delta(x, b), b)$$

$$F((x, a)) = \begin{cases} 1, x \in F, \\ 0, x \notin F \end{cases}$$



Построение из VPA



Построение с помощью сигналов



Инкрементальное обучение ТА



Прикладные задачи

- Быстрое распознавание сложных языков — например, палиндромы
- Быстрая разметка всех подслов с помощью ТА
- Эффективный поиск подслов, принадлежащих языку
- Лексический анализ



Лексический анализ

Пусть дано входное слово $w = w_1 \dots w_n$. $f : Q \rightarrow D \cup \{-1\}$ — функция разметки состояний ТА лексическими доменами.

1. Отображение входного слова в состояния ТА;
2. Инициализируется массив длины $|w|$, состоящий из пар (l, k) — длина и домен токена — значениями $(l_i, k_i) = (1, f(l(w_i)))$.
3. После k -го шага ТА получается последовательность состояний $s_1 \dots s_{n-k}$, лексические домены $k_i = f(s_i)$. Если домен k_i не равен -1 , то обновляется i -я ячейка массива значением $(k + 1, k_i)$.
4. После всех шагов ТА в массиве хранятся пары, описывающие самый длинный токен, начинающийся на позиции i , длиной l , принадлежащий лексическому домену k . Массив образует односвязный список последовательности токенов.



Пример лексического анализа

5	5	6	6	4	5	5				
1	5	5	6	6	4	5	5			
0	5	5	2	6	6	4	5	5		
1	4	1	2	2	6	3	4	1	2	
0	2	0	2	2	2	3	3	0	2	2

a b a b b b c c a b b

Результат разбиения на токены

$[(0, 3, abab, 1), (4, 5, bb, 2), (6, 7, cc, 3), (8, 9, ab, 1), (10, 10, b, 2)]$

...	...											
4	(4, 1)	(1, 2)	(2, 1)	(2, 2)	(2, 2)	(1, 2)	(2, 3)	(1, 3)	(2, 1)	(2, 2)	(1, 2)	
3	(4, 1)	(1, 2)	(2, 1)	(2, 2)	(2, 2)	(1, 2)	(2, 3)	(1, 3)	(2, 1)	(2, 2)	(1, 2)	
2	(2, 1)	(1, 2)	(2, 1)	(3, 2)	(2, 2)	(1, 2)	(2, 3)	(1, 3)	(2, 1)	(2, 2)	(1, 2)	
1	(2, 1)	(1, 2)	(2, 1)	(2, 2)	(2, 2)	(1, 2)	(2, 3)	(1, 3)	(2, 1)	(2, 2)	(1, 2)	
0	(1, -1)	(1, 2)	(1, -1)	(1, 2)	(1, 2)	(1, 2)	(1, 3)	(1, 3)	(1, -1)	(1, 2)	(1, 2)	
<i>kpos</i>	0	1	2	3	4	5	6	7	8	9	10	



Проблемы клеточных автоматов

Трудный анализ : Задачи пустоты, эквивалентности, полноты и минимизации для клеточных автоматов являются алгоритмически неразрешимыми $\Rightarrow$ отсутствие аналогов теоремы Майхилла-Нероде или леммы о накачке. Трудоемкость интерпретации состояний

Большой размер модели : Алгоритм построения ТА по ДКА $(Q, \Sigma, \delta, q_0, F)$ порождает клеточный автомат из $|\Sigma| \cdot |Q|$ состояний. Для линейно-конъюнктивной грамматики в LNF (Σ, N, P, S) — $|\Sigma|^2 \cdot 2^{|N|}$ состояний.

Ассимптотика вычислений клеточных автоматов realtime $O(n^2)$, lineartime — $O(kn^2)$

Для ДКА, VPA, DPDA — $O(n)$; LL, LR парсинг — $O(n)$; CYK парсинг — $O(n^3)$



Часть 3: А что делать то?



Первые предложения оптимизации

Гипотеза

Не все локальные правила и состояния используются ТА при распознавании слова

Правило $\delta(x, y)$ ТА достижимо, если $\exists w \in \Sigma^*$, для которого на предпоследнем шаге вычислений получается конфигурация xu . Первой идеей анализа достижимых стал поиск недостижимых состояний методом поиска в глубину (далее — $\mathcal{A}_0$). Показал слабую эффективность.

Вторая идея — поиск в глубину с выбором состояний, допустимых на очередном шаге вычислений (далее — $\mathcal{A}_1$). Дает более точную оценку



Алгоритм "домино"

Алгоритм Домино $\mathcal{A}_2$ как развитие идеи поиска в глубину

1. Инициализируется множество возможных пар стартовых символов $\{(l(x), l(y)) : x, y \in \Sigma\}$
2. $S_{i+1} = \{(\delta(x, y), \delta(y, z)) : (x, y), (y, z) \in S_i\};$
3. Останавливается $\exists i < j : S_i \subseteq S_j;$
4. Пара (x, y) недостижима, если $\nexists i : (x, y) \in S_i.$

Алгоритм можно продолжить на случаи подстрок длины $N - \mathcal{A}_N.$

A0

	0	1	2	3	4	5	6	7	8
0	0	0	0	3	3	5	1	1	1
1	7	7	7	3	3	5	6	6	6
2	8	8	8	5	5	5	8	8	8
3	7	7	7	3	3	5	6	6	6
4	2	2	2	5	5	5	2	2	2
5	5	5	5	5	5	5	5	5	5
6	7	7	7	3	3	5	6	6	6
7	0	0	0	3	3	5	1	1	1
8	8	8	8	5	5	5	8	8	8

A1

	0	1	2	3	4	5	6	7	8
0	0	0	0	3	3	5	1	1	1
1	7	7	7	3	3	5	6	6	6
2	8	8	8	5	5	5	8	8	8
3	7	7	7	3	3	5	6	6	6
4	2	2	2	5	5	5	2	2	2
5	5	5	5	5	5	5	5	5	5
6	7	7	7	3	3	5	6	6	6
7	0	0	0	3	3	5	1	1	1
8	8	8	8	5	5	5	8	8	8

A2

	0	1	2	3	4	5	6	7	8
0	0	0	0	3	3	5	1	1	1
1	7	7	7	3	3	5	6	6	6
2	8	8	8	5	5	5	8	8	8
3	7	7	7	3	3	5	6	6	6
4	2	2	2	5	5	5	2	2	2
5	5	5	5	5	5	5	5	5	5
6	7	7	7	3	3	5	6	6	6
7	0	0	0	3	3	5	1	1	1
8	8	8	8	5	5	5	8	8	8



Пример работы алгоритма "Домино"

Пример для ТА, распознающего язык $(a|b)^* b(a|b)^*$

$$I(a) = 0 \quad I(b) = 1$$

$S_0 :$

	0	1	2
0	0	1	0
1	2	1	2
2	2	1	2

$S_1 :$

	0	1	2
0	0	1	0
1	2	1	2
2	2	1	2

$S_2 :$

	0	1	2
0	0	1	0
1	2	1	2
2	2	1	2

- 0 — a^+
- 1 — $(a^*b)^+$
- 2 — $(a^*b)^+ a^+$

Показано, что правило $\delta(0, 2)$ недостижимо



Свойства алгоритмов "Домино"

Сходимость

Алгоритмы $\mathcal{A}_i$ для данного ТА $\mathcal{T}$ образуют сходящуюся за конечное число шагов последовательность множеств достижимых состояний

$$\mathcal{A}_0(\mathcal{T}) \supseteq \mathcal{A}_1(\mathcal{T}) \supseteq \mathcal{A}_2(\mathcal{T}) \supseteq \dots$$

Алгоритмы могут дать только ложноположительный ответ
Задача поиска достижимых правил эквивалентная задаче пустоты языка, поэтому также считается неразрешимой алгоритмически

Гипотеза

Алгоритм $\mathcal{A}_2$ дает точный результат для регулярных и линейных языков (случайным тестированием не были получены)



Алгебраический анализ ТА

Локальное правило ТА задает группоид над множеством состояний. Группоид — множество с определенной на нем бинарной операцией.

Отношение эквивалентности $\sim$ — конгруэнция, если $\forall a, b, x, y \in Q : a \sim b, x \sim y \Rightarrow \delta(a, x) \sim \delta(b, y)$.

Предложение

Пусть $(Q, \Sigma, \delta, h, F)$ — ТА, и отношение конгруэнции $\sim$ на группоиде (Q, δ) сохраняет разбиение на принимающие-отвергающие состояния ТА, то есть

$$x \sim y \Rightarrow (x \in F \wedge y \in F) \vee (x \notin F \wedge y \notin F)$$

Тогда отношение $\sim$ задает отношение эквивалентности на состояниях ТА.



Пример актуальности поиска достижимых правил

	0	1	2
0	0	1	0
1	2	1	2
2	2	1	2

	0	1
0	0	1
1	1	1

Для полной таблицы невозможно обосновать эквивалентность состояний $1 \sim 2$ из-за противоречия $\delta(0, 1) \neq \delta(0, 2)$. В то же время, если считать пару состояний $(0, 2)$ недостижимой, то доказательство возможно



Переход к частичному группоиду

Анализ полной таблицы часто показывает незначительный результат. Анализ достижимых операций группоида позволяет перейти от полного группоида к частично-определенному группоиду, у которого операция определена не для всех элементов. Пусть (Q, δ) — частичный группоид.

- Отношение эквивалентности $\sim_R$ называют правой конгруэнцией, если $\forall x, y, q \in Q$ либо $\delta(x, q)$ не определено, либо $\delta(y, q)$ не определено, либо $\delta(x, q) \sim_R \delta(y, q)$
- Отношение эквивалентности $\sim_L$ называют левой конгруэнцией, если $\forall x, y, q \in Q$ либо $\delta(q, x)$ не определено, либо $\delta(q, y)$ не определено, либо $\delta(q, x) \sim_L \delta(q, y)$
- Отношение эквивалентности $\sim_Q$ называют квазиконгруэнцией, если оно одновременно является левой и правой конгруэнцией
- Отношение эквивалентности $x \sim y$ называют конгруэнцией, если $\forall a, b, x, y \in Q : a \sim b, x \sim y \Rightarrow \delta(a, x) \sim \delta(b, y)$.



Свойства конгруэнции

$\text{Con } A$ — решетка конгруэнций, $Q \text{ Con } A$ — решетка квазиконгруэнций, $R_i \text{ Con } A$ — решетка конгруэнций на i -й позиции (левая или правая конгруэнция).

- $\text{Con } A \subseteq Q \text{ Con } A \subseteq R_i \text{ Con } A$
- Для полного группоида $Q \text{ Con } A = \text{Con } A$, это не так для частичного группоида
- Существует критерий, что квазиконгруэнция является конгруэнцией ²

²*Решетников А. В.* О конгруэнциях частичных n -арных группоидов. // Известия Саратовского университета. Новая серия. Серия Математика. Механика. Информатика. 2011. Т. 11, № 3—2. С. 46—51



Анализ конгруэнций "сверху-вниз"

1. Инициализируется первое приближение разбиения на классы эквивалентности
2. Выполняется проверка внутри класса эквивалентности:
$$\forall x, y \in C : \delta(x, x) = \delta(x, y) = \delta(y, x) = \delta(y, y)$$
3. Выполняется попарная проверка классов эквивалентности на наличие конфликтов в разбиении по определению конгруэнции
$$\forall x_1, x_2 \in C_x, y \in C_y : \delta(x_1, y) = \delta(x_2, y) \wedge \delta(y, x_1) = \delta(y, x_2)$$
4. При наличии конфликтов, класс эквивалентности разбивается на несколько других
5. Повторять, пока конфликтов не будет



Ограничение множества допустимых конгруэнций

1. На первом шаге допустимыми считаются конгруэнции

$$S_1 = \{(x, y) : x, y \in F\} \cup \{(x, y) : x, y \in Q \setminus F\}$$

2. Для каждой допустимой конгруэнции проверяем требуется ли она недопустимой конгруэнции, и отсеиваем все, которые требуют недопустимую:

$$S_{i+1} = \{(a, b) : (a, b) \in S_i, (\delta(a, a), \delta(b, b)) \in$$

$$S_i, (\delta(a, c), \delta(b, c)) \in S_i, (\delta(c, a), \delta(c, b)) \in S_i, \forall c \in Q\}$$

3. Повторять, пока $S_{i+1} \neq S_i$

	0	1	2	3	4	5	6	7	8
0	0	0	0	3	3	3	1	1	1
1	7	7	7	3	3	3	6	6	6
2	8	8	8	5	5	5	8	8	8
3	7	7	7	3	3	3	6	6	6
4	2	2	2	5	5	5	2	2	2
5	8	8	8	5	5	5	8	8	8
6	7	7	7	3	3	3	6	6	6
7	0	0	0	3	3	3	1	1	1
8	8	8	8	5	5	5	8	8	8



Пример поиска



Пример состояния-ловушки ТА



Состояния-ловушки

Состояние-ловушка — состояние ТА, появление которого в вычислении даёт немедленный ответ о принадлежности слова языку. **Пример:** состояния, характеризующие подслова, нарушающие регулярную структуру языка — состояния-ловушки. Состояние $q_1 \in Q$ может достичь состояния q_2 *слева (справа)*, если $q_1 = q_2$ или в строке (столбце) таблицы ТА есть состояние, которое может достичь q_2 слева (справа) с учетом определенности операции.

- **Необходимое условие:** если q состояние-ловушка, значит q не может достичь принимающего состояния слева или справа
- **Достаточное условие:** если состояние q вместе с состояниями, достижимыми слева и справа, порождает подалгебру, в которой нет принимающих состояний, тогда q — состояние-ловушка.
- **Оптимизация:** состояния-ловушки можно объединить в одно состояние f с локальным правилом $\forall x : \delta(x, f) = \delta(f, x) = f$



Ограничения алгоритма достижимости "домино"

	1	2	3	4	5	6	7	8
1	7	7	3	3	5	6	6	6
2	8	8	5	5	5	8	8	8
3	7	7	3	3	5	6	6	6
4	2	2	5	5	5	2	2	2
5	5	5	5	5	5	5	5	5
6	7	7	3	3	5	6	6	6
7	0	0	3	3	5	1	1	1
8	8	8	5	5	5	8	8	8

$$L = a^n b a^n$$

$$I(a) = 6$$

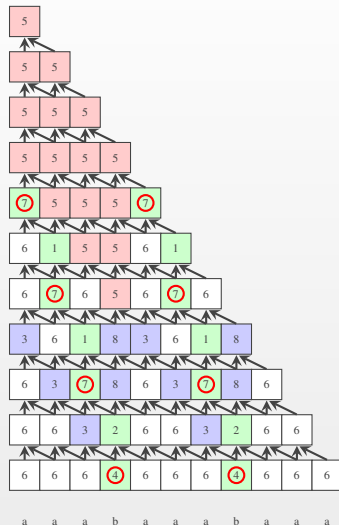
$$I(b) = 4$$

$$F = \{4, 7\}$$

$$\left. \begin{array}{l} 86^N - \text{достижимы} \\ 6^N 7 - \text{достижимы} \end{array} \right\} \Rightarrow 86^N 7 \text{ достижимы}$$

При анализе сигналов $86^N 7$ недостижимы
 $\Rightarrow$ алгоритм дал неверную оценку

На основе данного контрпримера был
 построен линейный язык, для которого
 алгоритмы $\mathcal{A}_N$ не дадут точного ответа
 $\{a^n b a^n\} \cup a^* b a^* b(a|b)^*$



ТА как конечные преобразователи

Определение

Конечным преобразователем называется действие автомата Мили на формальный язык, который действует на все слова, входящие в формальный язык, тем самым задает новый формальный язык.

Действие конечного преобразователя $A_T = (S, \Sigma, \Sigma, \delta_1, \lambda, S_0)$ на ДКА $A = (Q_A, \Sigma, \delta_2, q_0, F)$ порождает регулярный язык, описываемый НКА $(Q', \Sigma, \delta', q'_0, F')$ по правилам:

- $Q' = S \times Q_A$; $q'_0 = (S_0, q_0)$; $F' = \{(x, y) : x \in S, y \in F\}$;
- $\delta'((x, y), b) = \{(\delta_1(x, a), \delta_2(y, a)) : \lambda(x, a) = b\}$.

Треллис Автомат, как конечный преобразователь $(S_{FT}, X_{FT}, Y_{FT}, \delta_{FT}, \lambda_{FT}, S_{FT}^{(0)})$, определяется как

- $S_{FT} = Q' \cup \{[-]\}$; $X_{FT} = Y_{FT} = Q'$; $\delta_{FT}(x, q) = q$;
- $\lambda_{FT}([-], q) = \varepsilon$; $\lambda(x, q) = \delta'(x, q)$; $S_{FT}^{(0)} = [-]$.



Алгоритм точного анализа достижимых состояний

1. Инициализируется ДКА, распознающий язык слов из стартовых состояний
2. Строится ДКА, распознающий язык конфигураций следующего шага, с помощью действия ТА как конечный преобразователь;
3. Строится набор возможных пар для каждого шага вычислений;
4. Останавливается $\exists i < j : S_i \subseteq S_j$;
5. Пара (x, y) недостижима, если $\nexists i : (x, y) \in S_i$.

Проблема — более чем экспоненциальный рост



Теорема

Для ТА $M = M(A) = (\Sigma, Q', h, \delta', F')$, построенного на базе ДКА, $A = (Q, \Sigma, \delta, q_0, F)$, алгоритм $\mathcal{A}_2$ дает точный результат оценки достижимых пар состояний ТА.

Доказательство основано на сохранении вида ДКА "домино" — распознает слова состоящих из множества допустимых пар символов — при действии на него ТА как конечного преобразователя



Оптимизация интерпретации — ограничение области вычислений через разновидности состояний ловушек. Рассматривается оптимизация вычисления всей последовательности конфигураций, а не нахождения ответа о принадлежности строке. Тривиальные правила

- Если все не достигают слева принимающего — заканчиваем
- Если все не достигают справа принимающего — заканчиваем

Сложные правила

- Если в нескольких первых (последних) состояниях не достигают слева (справа) принимающего, то можно уменьшить область вычислений на эти состояния
- Если было получено состояние-ловушка, то можно разделить область вычислений на 2 отдельные и считать только их

БОЛЬШОЙ ПРИМЕР



Оптимизация правилом принятия слова

- Принятие слова, если все в конфигурации принимающие
- Принятие слова, если хоть одна в конфигурации принимающее



А это все?

Дальнейшие пути изучения

- Эквивалентность состояний по шагам
- Оптимизация вычислений по шагам
- Критерий состояния-ловушки
- Быстрые алгоритмы ограничения области вычислений
- Изучение связи оптимизаций с другими распознавателями



Часть 4: "Зоопарк" клеточных автоматов



История параллельных распознавателей

- 1966 — Iterative arrays — первая модель массового параллелизма распознавания формальных языков³
- 1970 — Показана эквивалентность клеточных автоматов, iterative arrays и линейно-ограниченных языков⁴
- 1972 — Распознавание языков клеточными автоматами реального времени
- 1980 — Однонаправленный клеточный автомат реального времени - простейший параллельный распознаватель⁵

³*Cole S.* Real-Time Computation by n-Dimensional Iterative Arrays of Finite-State Machines. // *IEEE Transactions on Computers*. 1969. Т. С—18, № 4. С. 349—365

⁴*Smith A. R.* Cellular automata and formal languages. 1970

⁵*Cole S.* Real-Time Computation by n-Dimensional Iterative Arrays of Finite-State Machines. // *IEEE Transactions on Computers*. 1969. Т. С—18, № 4. С. 349—365



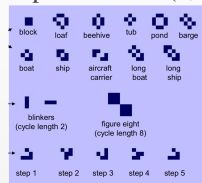
Клеточные автоматы

Определение

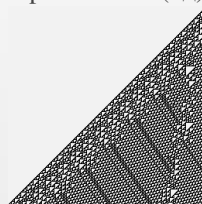
Клеточный автомат над полем размерности d определяется кортежем (S, c, N, f) , где

- S — конечное множество состояний;
- $c : \mathbb{Z}^d \rightarrow S$ — конфигурация клеточного автомата;
- $N = (x_1, \dots, x_n)$, $x_i \in \mathbb{Z}^d$ задает окрестность каждой точки пространства как набор $x + x_i$ для $i = 1, \dots, n$;
- локальная функция $f : S^n \rightarrow S$ определяет новое состояние клетки в позиции x на следующем временном шаге как $f(c(x + x_1), \dots, c(x + x_n))$.

Игра "Жизнь" (2Д)



Правило 110 (1Д)



Классификация одномерных КА

По направлению передачи информации

Однонаправленные

Двунаправленные

По времени вычисления

Реального времени

Линейного времени

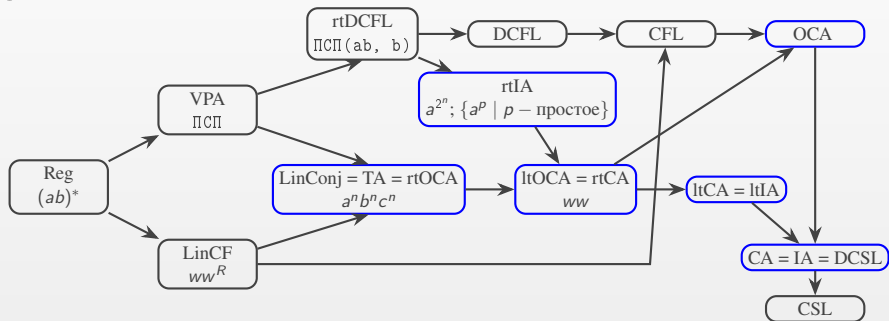
Сверхлинейного
времени



Отношения классов формальных языков

Классы языков

- Классическая иерархия формальных языков
- Клеточные автоматы



Свойства клеточных автоматов

Замкнутость классов языков

	Булевы операции ($L_1 \cup L_2, L_1 \cap L_2, \bar{L}$)	$L_1 L_2$	L^R	$h(L)$	$h_{code}(L)$	$h^{-1}(L)$
$rtOCA = TA$	+	-	+	-	+	+
$ltOCA = rtCA$	+					
$ltCA = ltIA$	+					
OCA	+					
CA	+					

